

SOFTWARE REQUIREMENTS SPECIFICATION

(SRS)

Smart Adaptive Career Roadmap Generator using Skill Gap Prediction

Document Version: 1.0

Prepared for: Project Guide Review — Final Year Major Project (Semester 7)

Date: June 2026

Field	Detail
Project Title	Smart Adaptive Career Roadmap Generator using Skill Gap Prediction
Document Type	Software Requirements Specification (SRS)
Methodology	Requirements derived from comparative study of 7 existing systems, mapped to current implementation
Primary Tech Stack	ASP.NET Core 8, PostgreSQL, React (Vite), Python FastAPI + scikit-learn
Status	Core system implemented; this document also defines remaining/planned scope

Table of Contents

(Right-click the field below and select "Update Field", or press F9 / Ctrl+A then F9, in Microsoft Word to populate page numbers.)

Table of Contents.....	2
1. Introduction	4
1.1 Purpose	4
1.2 Document Conventions.....	4
1.3 Intended Audience	4
1.4 Project Scope	4
1.5 References — Existing Systems Studied	4
2. Overall Description.....	6
2.1 Product Perspective	6
2.2 Product Features Summary	6
2.3 User Classes and Characteristics.....	6
2.4 Operating Environment	6
2.5 Design and Implementation Constraints	6
2.6 Assumptions and Dependencies	7
3. Comparative Analysis of Existing Systems	8
3.1 Comparison Matrix	8
3.1.1 Reference Screenshots.....	8
3.2 Common Functionalities Identified	17
3.3 Identified Gaps and Limitations.....	17
3.4 This System's Differentiation.....	18
4. Functional Requirements	18
4.1 User Account Management.....	18
4.2 Skill Catalog & Profile Management	19
4.3 Career Role Catalog.....	19
4.4 Skill-Gap Analysis Engine	19
4.5 Roadmap Generation & Tracking	20
4.6 Employability Scoring.....	20
4.7 AI/ML-Powered Recommendation Engine.....	20
4.8 Dashboard & Visualization	21
4.9 Learning Resources (Planned Extension)	21
4.10 Administration (Planned Extension).....	21
5. Non-Functional Requirements	23
5.1 Performance	23
5.2 Security.....	23

5.3 Usability	23
5.4 Reliability & Availability	23
5.5 Scalability	24
5.6 Maintainability	24
5.7 Portability	24
5.8 Compliance & Data Privacy	24
6. External Interface Requirements	26
6.1 User Interfaces	26
6.2 Software Interfaces	26
6.3 Communication Interfaces	26
7. System Architecture Overview	27
7.1 Layer Responsibilities	27
8. Tools & Technology Stack	28
8.1 Frontend Technologies	28
8.2 Backend Technologies	28
8.3 Database & Data Layer	28
8.4 Machine Learning Service	29
8.5 Development & Testing Tools	29
8.6 Hardware Requirements (Development Environment)	29
9. Data Requirements (Database Schema Summary)	31
10. Appendix	32
10.1 Glossary	32
10.2 Full References	32

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements for the Smart Adaptive Career Roadmap Generator. Rather than listing requirements arbitrarily, this document derives them from a structured comparative study of seven existing career-guidance and skill-gap platforms (Section 3), cross-referenced against the system already implemented, and extended with new capabilities identified as gaps in the existing market. It serves as a reference for the development team, the project guide, and project evaluators.

1.2 Document Conventions

Functional requirements are labeled FR-1, FR-2, and so on, grouped by module. Non-functional requirements are labeled NFR-1, NFR-2, and so on, grouped by quality attribute. Each requirement is marked with a Status of either Implemented (already built and tested in the current system) or Planned (identified as necessary but not yet built), so the document doubles as an honest progress tracker.

1.3 Intended Audience

- Project guide / evaluator — to assess scope, completeness, and technical depth
- Development team (self) — as a build reference for remaining work
- Future contributors — as onboarding documentation for system scope and architecture

1.4 Project Scope

The system analyzes a user's existing skills, education, and career aspirations to identify the gap between their current competencies and a target job role's requirements. It then generates a personalized, ordered learning roadmap to close that gap, tracks the user's progress against it, and computes an employability score — both through transparent rule-based weighting and through a trained machine learning model — to indicate readiness for the target role. The system additionally recommends alternative roles the user is already well-positioned for, based on skill similarity.

1.5 References — Existing Systems Studied

The following seven existing systems were reviewed to identify common functionality, design patterns, and unmet gaps before defining the requirements in this document. Short codes (S1–S7) are used to trace individual requirements back to the systems that informed them.

[S1] LinkedIn Learning — Career Explorer & Skill Assessments — <https://www.linkedin.com/learning/>

[S2] Coursera Career Academy — <https://www.coursera.org/career-academy>

[S3] Career.io — Career Pathways — <https://career.io/career-pathways>

[S4] roadmap.sh — Developer Learning Roadmaps — <https://roadmap.sh>

[S5] Resumly — Skills Gap Analyzer — <https://www.resumly.ai/skills-gap-analyzer>

[S6] Prosumely — Career Roadmap Generator & Skill Gap Analyzer — <https://www.prosumely.com/career-tools/career-roadmap-generator>

[S7] Fuel50 / iMocha — Enterprise Career Pathing Tools — <https://www.quantumworkplace.com/future-of-work/best-career-pathing-tools>

2. Overall Description

2.1 Product Perspective

The system is a standalone, web-based application built on a layered architecture: a React single-page frontend, an ASP.NET Core Web API as the central backend, a PostgreSQL relational database, and an independent Python-based machine learning microservice consulted by the backend for predictive features. It is not dependent on any third-party career platform and is designed so the ML layer can be retrained or replaced without changing the API contract.

2.2 Product Features Summary

- User registration, authentication, and profile management
- Master catalogs of skills and career roles, with role-to-skill importance mapping
- Skill-gap analysis between a user's current skills and a target role
- Automatic, personalized roadmap generation ordered by skill importance
- Step-level progress tracking with live completion percentage
- Dual employability scoring: rule-based (transparent, weighted) and ML-based (predictive)
- AI-driven learning-time estimation and alternative role recommendation
- Visual, animated dashboard summarizing all of the above

2.3 User Classes and Characteristics

User Class	Characteristics
Student / Early-career user	Primary user. Limited current skills, exploring multiple target roles, needs clear step-by-step guidance.
Working professional	Has existing skills and experience; uses the system primarily for upskilling toward a promotion or pivot.
Administrator (planned)	Manages the master skill and role catalogs; not yet exposed via UI — currently managed via direct API/database access.

2.4 Operating Environment

- Client: any modern desktop or mobile web browser (Chrome, Edge, Firefox)
- Backend: .NET 8 runtime, Windows or Linux server
- ML Service: Python 3.10+ runtime, deployable independently of the .NET backend
- Database: PostgreSQL 14+

2.5 Design and Implementation Constraints

- Backend framework is fixed as ASP.NET Core (Web API) per project proposal
- Database is fixed as PostgreSQL per project proposal
- Frontend is built with React, communicating with the backend exclusively over REST/JSON
- ML models are trained on a hybrid of curated real-world reference data (skill demand, role salary) and synthetic data (individual learning behavior), as real longitudinal user outcome data

is not available to a student project — this is disclosed transparently rather than presented as fully real-world-trained

2.6 Assumptions and Dependencies

- Users provide honest self-assessment of their proficiency levels when adding skills
- The ML microservice is reachable by the backend over HTTP; if unavailable, the system degrades gracefully to rule-based scoring only
- Skill and role catalogs are seeded/maintained centrally; end users do not create new skills or roles themselves in the current scope

3. Comparative Analysis of Existing Systems

Before defining requirements, the seven systems listed in Section 1.5 were evaluated against the core capabilities relevant to this project: skill-gap analysis, personalized roadmap generation, progress tracking, AI/ML-based prediction, and grounding in real market data. The matrix below summarizes this evaluation.

3.1 Comparison Matrix

System	Skill-Gap Analysis	Personalized Roadmap	Progress Tracking	AI/ML-Based	Market Data Grounding	Key Limitation Observed
S1 — LinkedIn Learning	Yes	Partial	Yes	Yes	Yes	Course-centric; no generated skill roadmap or employability score
S2 — Coursera Career Academy	Partial	Partial	Yes	Partial	Yes	Built around certificates/courses, not a skill-level gap engine
S3 — Career.io	Yes	Yes	Partial	Partial	Yes	Broad career exploration; limited individual skill-tracking depth
S4 — roadmap.sh	No	Partial	Yes	No	No	Static, generic roadmaps — not personalized to the individual's profile
S5 — Resumly	Yes	Yes	No	Yes	Yes	One-time analysis; no persistent account or progress dashboard
S6 — Prosumely	Yes	Yes	No	Yes	Yes	Similar one-shot generator; no ongoing score history or tracking
S7 — Fuel50 / iMocha	Yes	Yes	Yes	Yes	Partial	Enterprise (B2B) focused; not designed for individual student access

3.1.1 Reference Screenshots

The placeholders below mark where to paste a screenshot captured from each system, illustrating the feature referenced in the comparison matrix above. Paste directly over each bracketed line, then add a one-line caption underneath (Word: right-click image → Insert Caption).

[S1] LinkedIn Learning

Paste screenshot here — showing: skill assessment results or a Role Guide / Learning Path page

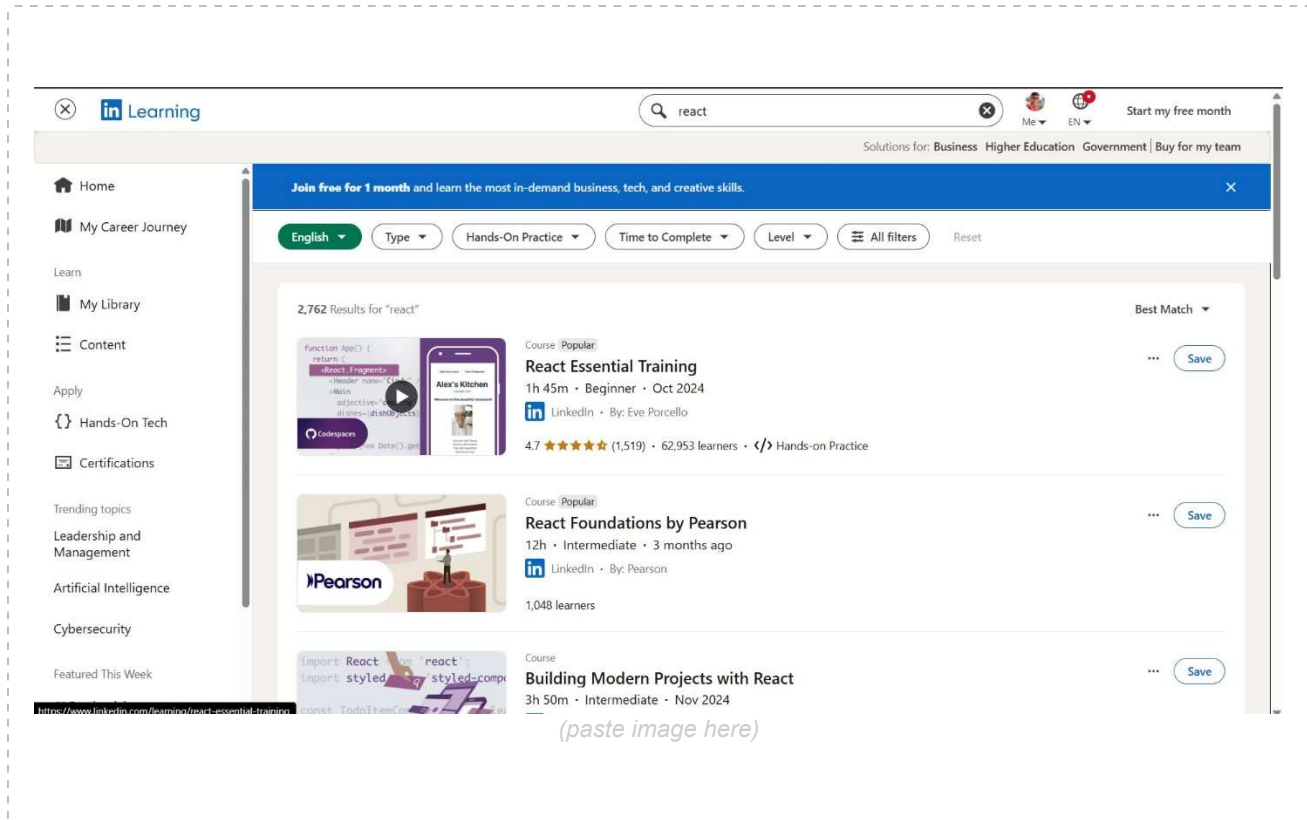


Figure 3.1 — LinkedIn Learning (S1)

[S2] Coursera Career Academy

Paste screenshot here — showing: role cards showing salary / job-demand data

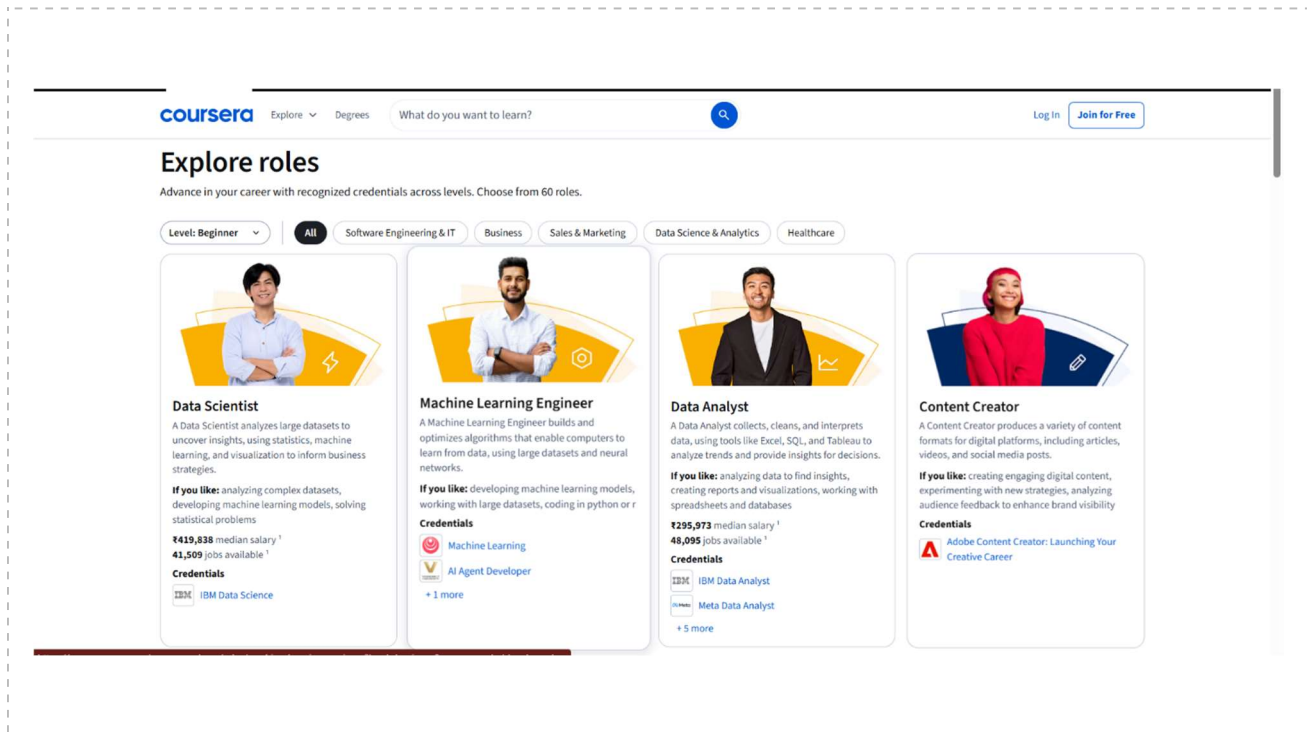


Figure 3.2 — Coursera Career Academy (S2)

[S3] Career.io

Paste screenshot here — showing: the Career Pathways skill-gap visual

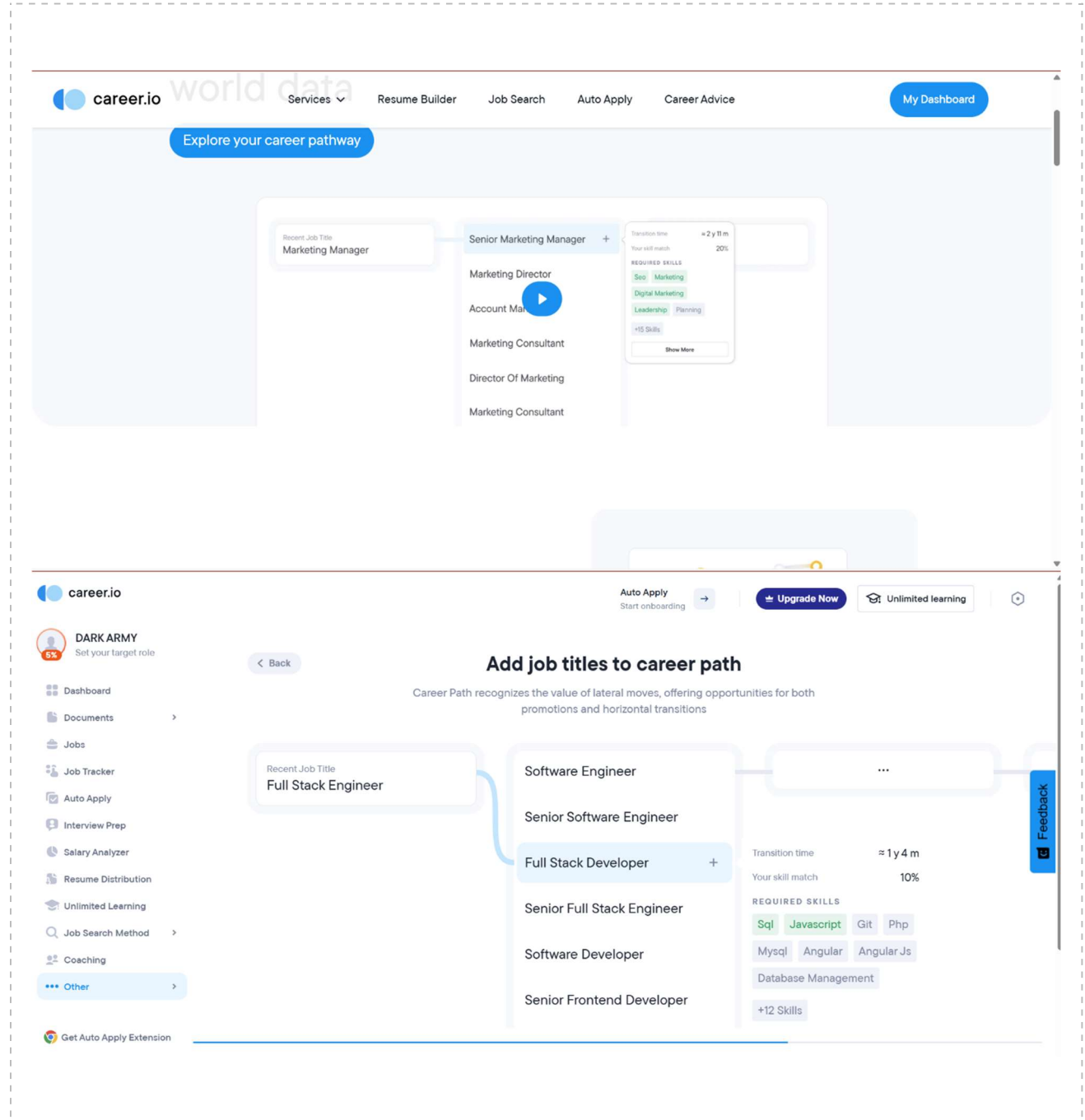
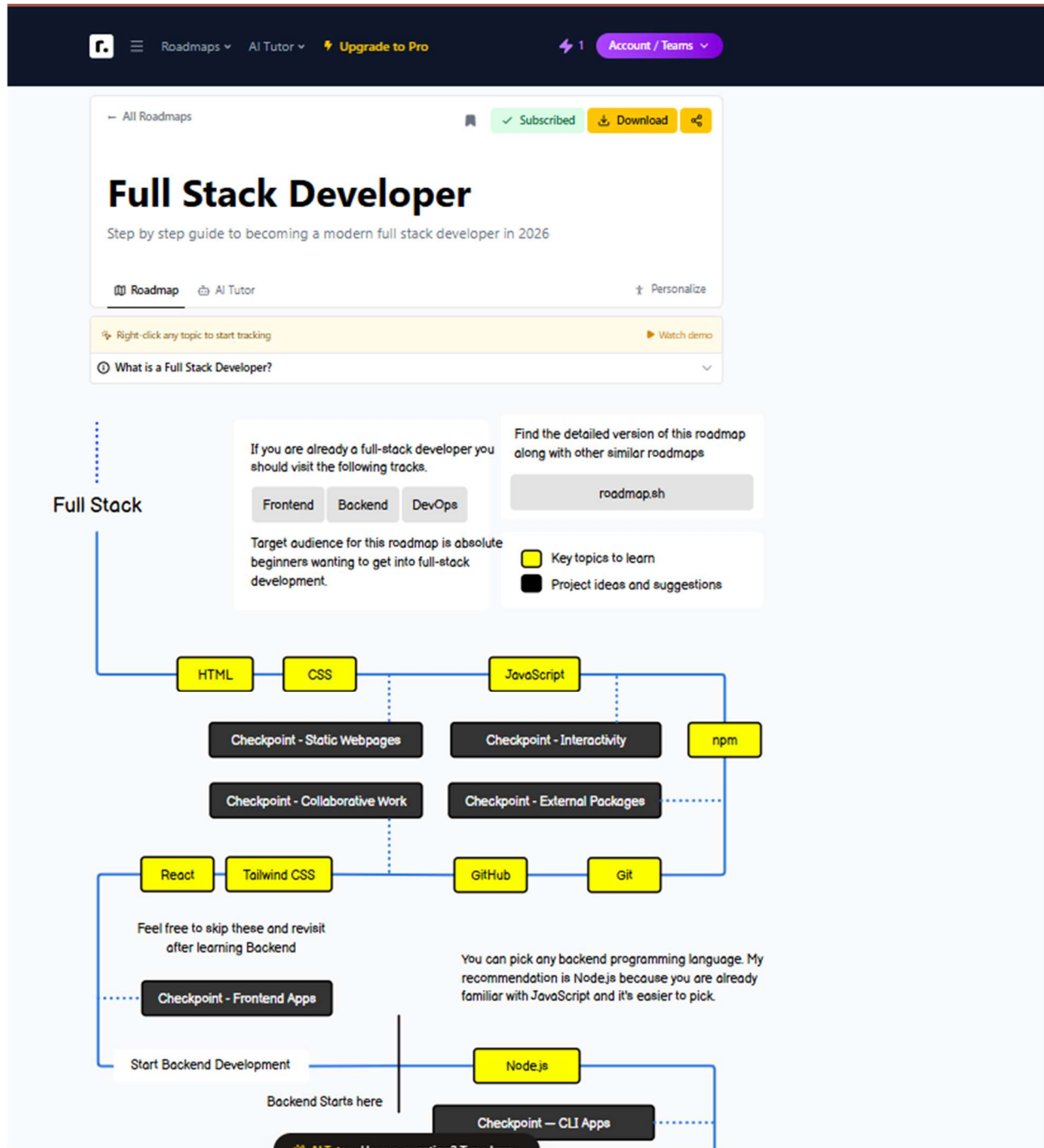


Figure 3.3 — Career.io (S3)

[S4] roadmap.sh

Paste screenshot here — showing: a role roadmap flowchart (e.g. Frontend Developer)



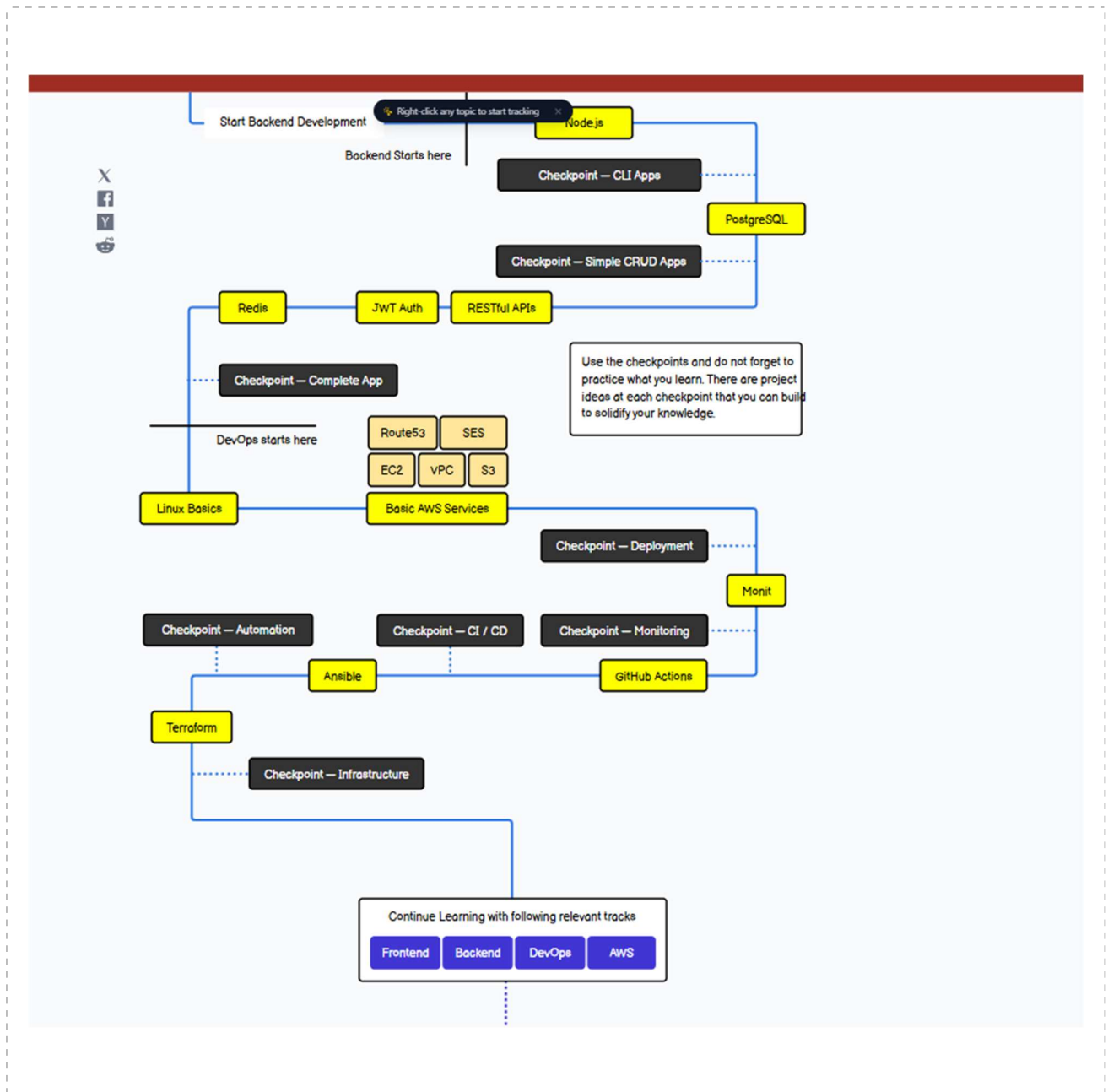


Figure 3.4 — roadmap.sh (S4)

[S5] Resumly

Paste screenshot here — showing: the Skills Gap Analyzer example results

(pa

See What Your Report Looks Like

This is a real sample report generated by our AI. Upload your resume above to get your own personalized analysis.

Overall Match
71%

Coverage
80%

Depth
70%

Recency
85%

ATS Keyword Match
88%

Detected Skills

Python

Engineering • Core

0.5y

Proficiency: 5/5

e.g., "Programming & Scripting: Python, SQL, Spark"

SQL

Engineering • Core

0.5y

Proficiency: 5/5

e.g., "Programming & Scripting: Python, SQL, Spark"

Spark

Engineering • Tool

1y

Proficiency: 4/5

e.g., "Programming & Scripting: Python, SQL, Spark"

Databricks

Engineering • Tool

0.8y

Proficiency: 4/5

e.g., "Data Engineering & Storage: Databricks, Snowflake, Airflow, Azure, AWS"

Snowflake

Engineering • Tool

0.8y

Proficiency: 4/5

e.g., "Data Engineering & Storage: Databricks, Snowflake, Airflow, Azure, AWS"

Airflow

Engineering • Tool

1y

Proficiency: 4/5

e.g., "Data Engineering & Storage: Databricks, Snowflake, Airflow, Azure, AWS"

Azure

Ops • Cloud

0.5y

AWS

Ops • Cloud

0.5y

Docker

Ops • Tool

1y

Target Profile Requirements

Python **Must-Have**

Req 4 You 5

AWS **Must-Have**

Req 4 You 5

MLOps **Must-Have**

Req 4 You 3

Leadership **Must-Have**

Req 5 You 5

Project Management **Must-Have**

Req 4 You 5

Scikit-Learn

Req 4 You 5

Power BI

Req 4 You 5

Peer baseline: Coverage median 78% • Depth median 68%

Coverage by Domain



Proficiency Profile



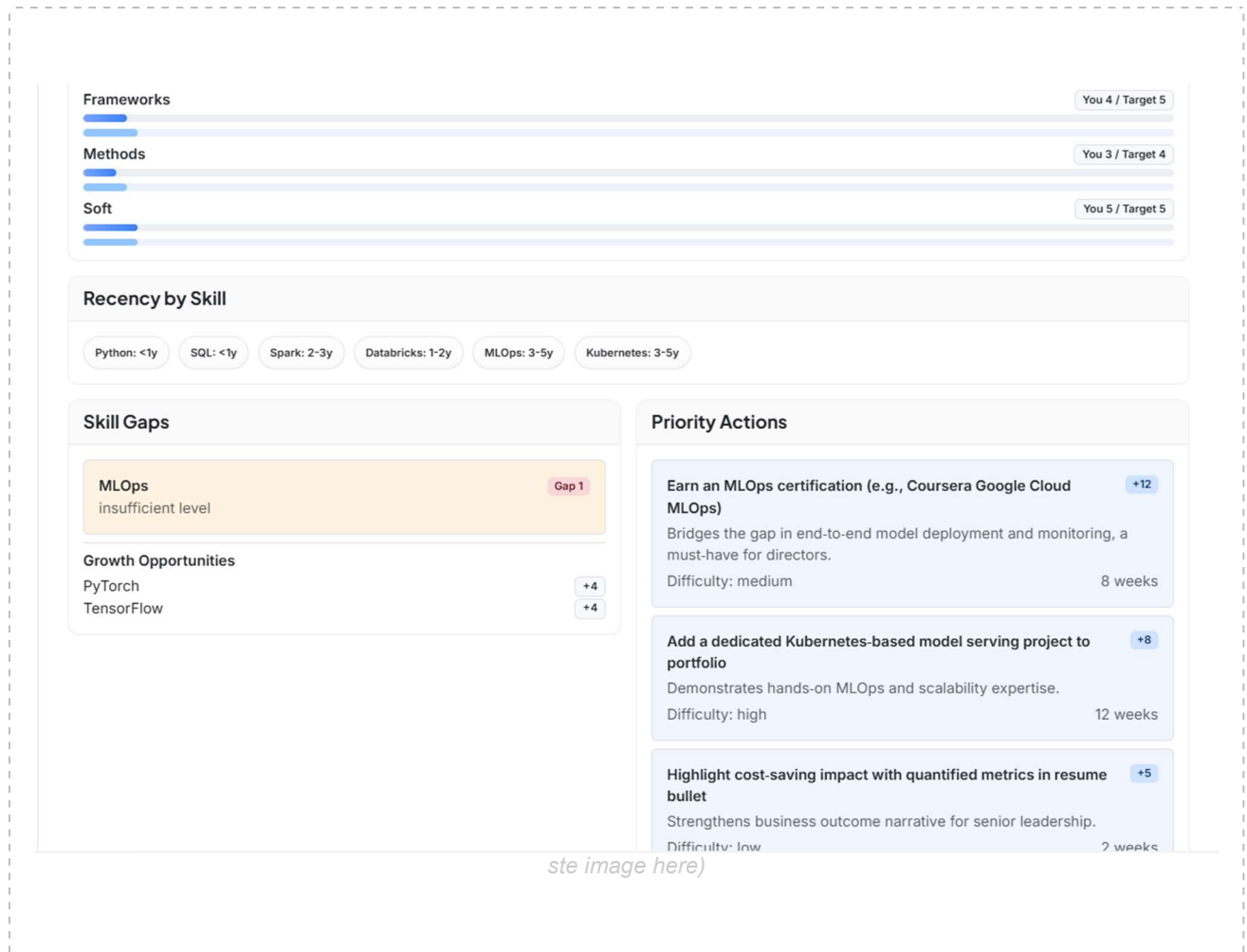
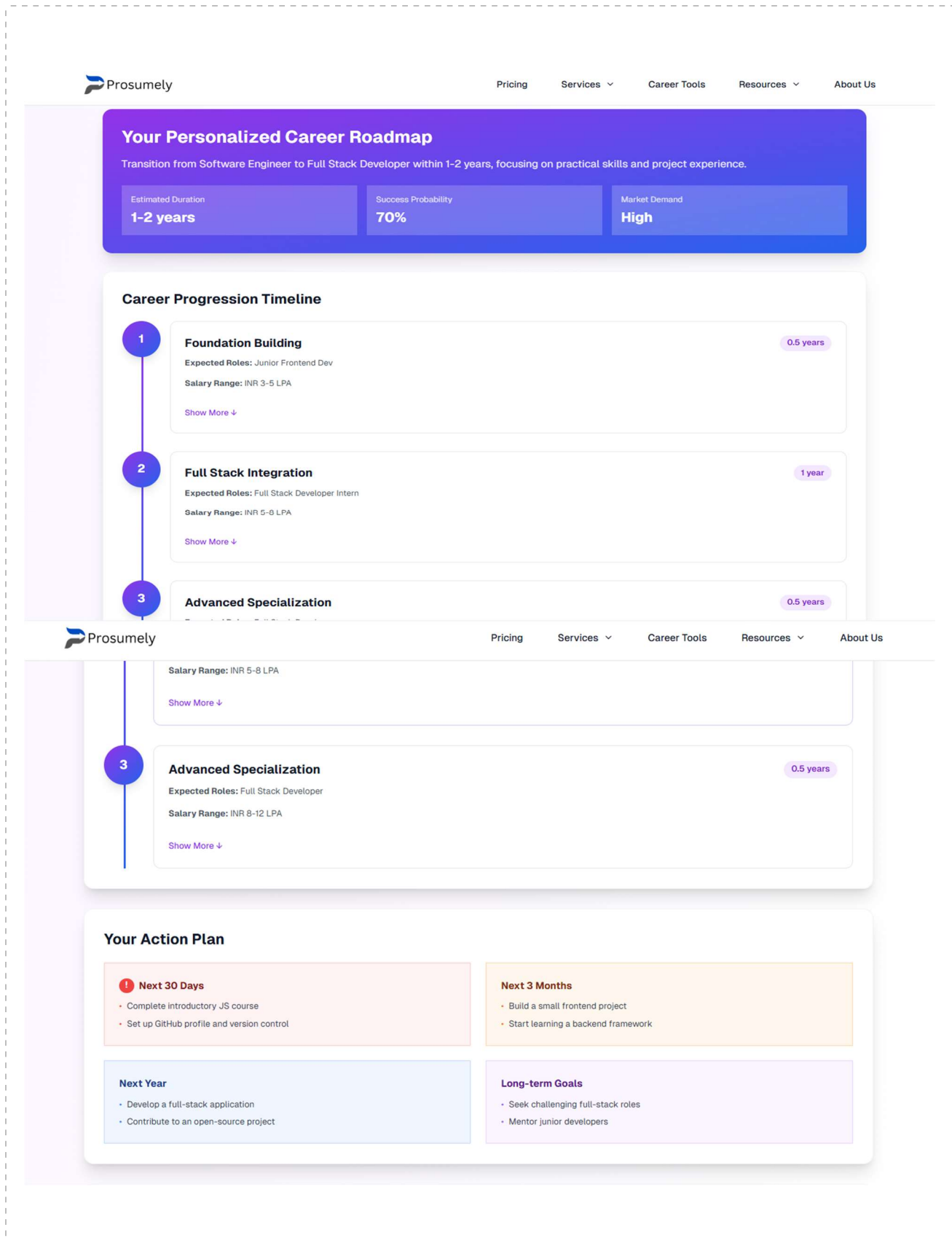


Figure 3.5 — Resumly (S5)

[S6] Prosumely

Paste screenshot here — showing: a sample generated Career Roadmap output



Skills Development Path

Frontend Skills

JavaScript (ES6+)

Resources: MDN Web Docs, freeCodeCamp

high priority3mo

React.js

Resources: React Docs, Scrimba

high priority3mo

Backend Skills

Node.js & Express.js

Resources: Node.js Docs, Udemy courses

high priority4mo

SQL Databases (PostgreSQL)

Resources: PostgreSQL Docs, SQLZoo

medium priority2mo

Alternative Career Paths

Frontend Specialist

Focus solely on advanced frontend development.

Duration: 1y

Pros:

Cons:

✓ Deep expertise

✓ High demand

✗ Limited backend scope

Backend Specialist

Specialize in server-side logic and databases.

Duration: 1y

Pros:

Cons:

✓ Strong problem-solving

✓ Scalability focus

✗ Less UI interaction

DevOps Engineer

Focus on infrastructure, deployment, and automation.

Duration: 1.5y

Pros:

Cons:

✓ Critical role

✓ High impact

✗ Steeper learning curve

Risks & Reality Check

Skill Obsolescence

Impact: Requires continuous learning.

Mitigation: Regularly update skills, follow tech trends.

medium likelihood

Burnout

Impact: Aggressive timeline can be draining.

Mitigation: Maintain work-life balance, take breaks.

high likelihood

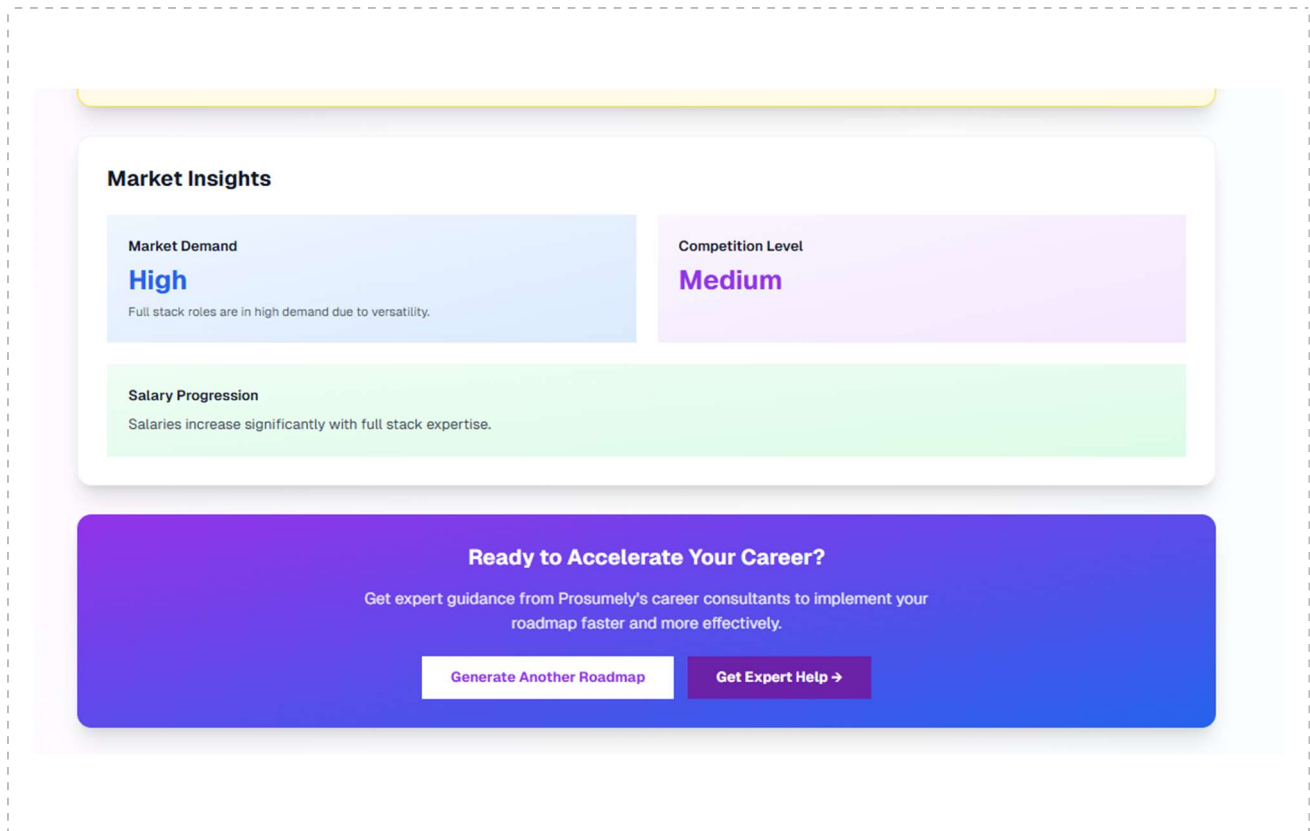


Figure 3.6 — Prosumely (S6)

3.2 Common Functionalities Identified

Across the majority of systems reviewed, the following functional pattern recurs and is treated as a baseline expectation for this project:

- Capturing the user's current skills/profile as an input
- Comparing current skills against a target role's required skills
- Producing some form of prioritized list of missing skills
- Generating a learning plan or roadmap from that list
- Grounding recommendations in market/demand data of some kind

3.3 Identified Gaps and Limitations

No single reviewed system combines all of the following, which the matrix above highlights as a recurring trade-off:

- Free, individual-student access (S7 is enterprise-only; most others are commercial SaaS tools)
- A persistent account with a roadmap that can be tracked over time, step by step (S5, S6 are one-time generators with no follow-up dashboard)
- Personalization driven by an actual stored skill profile rather than a generic template (S4 is the same roadmap for every visitor)
- Transparency between a rule-based score and a separate, explainable ML-predicted score shown side-by-side — none of the seven systems exposes this distinction to the user

- An open architecture (own database, own ML models) rather than a black-box third-party recommendation engine

3.4 This System's Differentiation

Building on the common pattern (Section 3.2) while directly addressing the gaps above (Section 3.3), this system is designed to provide:

- A persistent, account-based dashboard — not a one-time report — with historical score tracking (addresses gap in S5, S6)
- Step-level progress tracking with a live completion percentage tied directly to the generated roadmap (addresses gap in S5, S6; improves on S4's lack of personalization)
- Two side-by-side employability scores — a transparent rule-based weighted score and a trained ML-predicted score — so the user (and evaluator) can see both methods and how they compare (novel; not found explicitly in any of the seven systems)
- A similarity-based “other roles you're close to” recommendation, reusing the user's existing skill profile rather than asking them to repeat input (lighter-weight version of S7's enterprise role-matching, built for individuals)
- A fully open, self-built architecture (own PostgreSQL schema, own trained models) suitable for future extension and research, rather than a closed third-party engine

4. Functional Requirements

Requirements are grouped by module below. The “Derived From” column traces each requirement to the system(s) in Section 1.5 where the comparable feature was observed, or marks it as a new contribution of this project. The “Status” column reflects the actual current state of the implementation.

4.1 User Account Management

ID	Functional Requirement Description	Derived From	Status
FR-1	The system shall allow new users to register with full name, email, password, phone number, education level, and field of study.	S1, S2, S3, S5, S6	Implemented
FR-2	The system shall securely hash passwords (BCrypt) before storing them; plaintext passwords shall never be persisted.	Industry best practice	Implemented
FR-3	The system shall authenticate users via email and password, issuing a signed JWT access token on success.	S1, S2, S3, S5, S6, S7	Implemented
FR-4	The system shall restrict all data-modifying and personalized endpoints to requests carrying a valid JWT token.	Industry best practice	Implemented
FR-5	The system shall allow a logged-in user to update their profile information (education, field of study, experience).	S1, S3	Planned

ID	Functional Requirement Description	Derived From	Status
FR-6	The system shall support password reset via a verified email link.	S1, S2, S5	Planned

4.2 Skill Catalog & Profile Management

ID	Functional Requirement Description	Derived From	Status
FR-7	The system shall maintain a master catalog of skills, each categorized by domain (e.g., Frontend, Backend, AI/ML, Soft Skills).	S1, S2, S3, S4, S5, S6, S7	Implemented
FR-8	The system shall allow a user to add a skill they currently possess, along with a self-rated proficiency level (Beginner / Intermediate / Advanced / Expert).	S1, S3, S5, S6, S7	Implemented
FR-9	The system shall allow a user to remove a previously added skill from their profile.	S1, S3, S7	Implemented
FR-10	The system shall prevent duplicate skill entries for the same user.	Data integrity requirement	Implemented
FR-11	The system shall allow a user to import skills in bulk by uploading a resume for automatic parsing.	S5	Planned (new idea)

4.3 Career Role Catalog

ID	Functional Requirement Description	Derived From	Status
FR-12	The system shall maintain a catalog of career roles, each with industry, description, average salary, and demand level.	S1, S2, S3, S7	Implemented
FR-13	The system shall map each career role to its required skills, each tagged with an importance level (Must Have / Required / Good to Have / Optional).	S1, S3, S5, S6, S7	Implemented
FR-14	The system shall allow users to browse and explore all available career roles and their requirements.	S1, S2, S3, S4	Implemented
FR-15	The system shall allow users to filter or search roles by industry or demand level.	S2, S3	Planned

4.4 Skill-Gap Analysis Engine

ID	Functional Requirement Description	Derived From	Status
FR-16	The system shall compare a user's current skill set against a selected target role's required skills on demand.	S1, S2, S3, S5, S6, S7	Implemented
FR-17	The system shall produce the list of missing ("gap") skills, ordered by importance level (Must Have first).	S5, S6, S7	Implemented

ID	Functional Requirement Description	Derived From	Status
FR-18	The system shall detect when a user already possesses all required skills for a role and report role-readiness instead of generating an empty gap.	New (edge-case handling)	Implemented

4.5 Roadmap Generation & Tracking

ID	Functional Requirement Description	Derived From	Status
FR-19	The system shall automatically generate a personalized, ordered learning roadmap from the identified skill gap for a chosen role.	S3, S5, S6 (improves on static S4)	Implemented
FR-20	The system shall estimate the number of days required to learn each missing skill, based on its importance level.	New	Implemented
FR-21	The system shall allow a user to view all roadmaps they have previously generated.	S1, S3	Implemented
FR-22	The system shall allow a user to mark each roadmap step as Not Started, In Progress, or Completed.	S4, S7 (gap in S5, S6)	Implemented
FR-23	The system shall calculate and display the percentage of roadmap completion in real time as steps are updated.	S7	Implemented
FR-24	The system shall allow a user to attach a personal note to an individual roadmap step.	New	Planned (schema ready)
FR-25	The system shall notify a user when a roadmap step has remained “In Progress” for an extended period without completion.	New	Planned

4.6 Employability Scoring

ID	Functional Requirement Description	Derived From	Status
FR-26	The system shall calculate a rule-based, importance-weighted employability score (0–100%) for a user against a target role.	S5, S6, S7	Implemented
FR-27	The system shall classify the computed score into a readiness level: Job Ready, Almost Ready, Needs Improvement, or Beginner Stage.	S5, S6	Implemented
FR-28	The system shall store every calculated employability score with a timestamp, enabling score-over-time tracking.	New (gap in S5, S6)	Implemented
FR-29	The system shall additionally compute an ML-predicted employability score using a trained regression model, displayed alongside the rule-based score for transparency.	Novel — not found explicitly in S1–S7	Implemented

4.7 AI/ML-Powered Recommendation Engine

ID	Functional Requirement Description	Derived From	Status
FR-30	The system shall predict the estimated number of days a specific user will need to learn an average new skill, using a trained ML model based on years of experience, education level, and current skill count.	S1 (basic personalization), extended with ML	Implemented
FR-31	The system shall recommend alternative career roles the user is already closest to, based on skill-overlap similarity, excluding their currently selected target role.	S7 (enterprise role-matching, adapted for individuals)	Implemented
FR-32	The system shall allow its ML models to be retrained on updated data without requiring changes to the API contract.	Engineering best practice	Implemented (joblib-serialized, hot-swappable)
FR-33	The system shall support a “what-if” simulation showing the projected employability score increase if a specific missing skill were learned next.	New idea	Planned

4.8 Dashboard & Visualization

ID	Functional Requirement Description	Derived From	Status
FR-34	The system shall display a summary dashboard showing employability score, active roadmap count, skills added, and remaining skill-gap count.	S1, S3, S7	Implemented
FR-35	The system shall visually represent the employability score as an animated circular progress indicator.	New (presentation layer)	Implemented
FR-36	The system shall visually represent roadmap completion as an animated horizontal progress bar.	S7	Implemented
FR-37	The system shall clearly and separately label rule-based output versus ML-predicted output wherever both are shown, to preserve transparency.	Novel	Implemented

4.9 Learning Resources (Planned Extension)

ID	Functional Requirement Description	Derived From	Status
FR-38	The system shall maintain a catalog of learning resources (videos, articles, courses) mapped to individual skills.	S1, S2 (course catalogs)	Schema implemented; feature pending
FR-39	The system shall recommend specific learning resources for each step of a generated roadmap.	S1, S2 (adapted to tie into a personal roadmap, which S1/S2 do not do)	Planned

4.10 Administration (Planned Extension)

ID	Functional Requirement Description	Derived From	Status
FR-40	The system shall provide an administrator interface to manage the master skill catalog and career role catalog without direct database/API access.	S7 (lightweight, individual-scale equivalent)	Planned

5. Non-Functional Requirements

Non-functional requirements define the quality attributes the system must satisfy, independent of specific features.

5.1 Performance

ID	Category	Requirement	Status
NFR-1	Performance	Standard CRUD API responses (skills, roles, user data) shall return within 500ms under normal load.	Met in current testing
NFR-2	Performance	The full roadmap-generation and ML-prediction round trip shall complete within 2 seconds end-to-end.	Met in current testing

5.2 Security

ID	Category	Requirement	Status
NFR-3	Security	All user passwords shall be stored using one-way hashing (BCrypt); plaintext shall never be persisted or logged.	Implemented
NFR-4	Security	All endpoints except registration and login shall require a valid JWT bearer token.	Implemented
NFR-5	Security	All client-server and server-to-ML-service communication shall be capable of running over HTTPS/TLS in production.	Planned for deployment
NFR-6	Security	The system shall not expose raw database exceptions or stack traces to end users in a production build.	Planned (currently visible in Development mode)

5.3 Usability

ID	Category	Requirement	Status
NFR-7	Usability	The interface shall be usable by non-technical users through clear labels, color coding, and visual progress feedback.	Implemented
NFR-8	Usability	Any user action taking longer than 300ms shall display a loading or progress indicator.	Implemented
NFR-9	Usability	The interface shall adapt cleanly to desktop, tablet, and mobile viewport sizes.	Planned (responsive pass pending)

5.4 Reliability & Availability

ID	Category	Requirement	Status
NFR-10	Reliability	If the ML microservice is unreachable, core functionality (dashboard, roadmap, rule-based scoring) shall continue to operate without failure.	Implemented (graceful degradation)
NFR-11	Reliability	The database shall enforce referential integrity through foreign-key constraints and defined cascade rules.	Implemented

5.5 Scalability

ID	Category	Requirement	Status
NFR-12	Scalability	The database schema shall accommodate growth in users, skills, and roles without structural redesign.	Implemented (normalized 10-table schema)
NFR-13	Scalability	The ML microservice shall be independently deployable and scalable from the main backend, communicating only via REST.	Implemented

5.6 Maintainability

ID	Category	Requirement	Status
NFR-14	Maintainability	The backend codebase shall follow a layered architecture (Models, Data, Controllers, Helpers) to separate concerns.	Implemented
NFR-15	Maintainability	ML models shall be retrainable and hot-swappable (via serialized files) without modifying the API contract.	Implemented

5.7 Portability

ID	Category	Requirement	Status
NFR-16	Portability	The backend shall be deployable on any platform supporting the .NET 8 runtime and PostgreSQL.	Implemented
NFR-17	Portability	The ML service shall run on any platform supporting Python 3.10+, independent of the OS used for the .NET backend.	Implemented

5.8 Compliance & Data Privacy

ID	Category	Requirement	Status
NFR-18	Data Privacy	User passwords and personally identifiable data shall never be written to application logs in plaintext.	Implemented
NFR-19	Transparency	The system shall disclose to users/evaluators that ML predictions are trained on a hybrid of	Documented (Section 2.5)

ID	Category	Requirement	Status
		curated reference data and synthetic data, pending availability of real longitudinal user data.	

6. External Interface Requirements

6.1 User Interfaces

The system exposes a single web-based, responsive interface built in React, covering: Registration, Login, Dashboard, Skills management, Career Role browsing, and Roadmap detail/tracking views. No native mobile or desktop application is in scope.

6.2 Software Interfaces

The React frontend communicates exclusively with the ASP.NET Core backend via REST/JSON over HTTP. The ASP.NET Core backend, in turn, communicates with the Python ML microservice via REST/JSON over HTTP, and with PostgreSQL via Entity Framework Core (Npgsql provider). The frontend never calls the ML microservice directly.

6.3 Communication Interfaces

All API communication uses JSON payloads over HTTP/HTTPS. Authenticated requests carry a JWT bearer token in the Authorization header. Cross-Origin Resource Sharing (CORS) is configured on the backend to permit only the known frontend origin.

7. System Architecture Overview

The system follows a 4-layer architecture. The ASP.NET Core API Gateway is the single entry point for the React frontend, and internally communicates with both the PostgreSQL database and the Python ML Microservice. The ML Microservice does not expose any endpoint directly to the frontend — every request is brokered through the backend.

Client (React)	API Gateway (ASP.NET Core)	ML Microservice (Python FastAPI)	Database (PostgreSQL)
----------------	----------------------------	----------------------------------	-----------------------

Figure 7.1 — High-level architecture layers. The API Gateway calls the ML Microservice and Database independently; the frontend only ever talks to the API Gateway.

7.1 Layer Responsibilities

Layer	Responsibility
Client (React)	Renders UI, manages local session state, sends authenticated REST requests
API Gateway (ASP.NET Core)	Business logic, authentication, validation, database access, orchestrates calls to the ML service
ML Microservice (Python)	Hosts trained models; exposes prediction/recommendation endpoints consumed only by the API Gateway
Database (PostgreSQL)	Persists all structured data: users, skills, roles, roadmaps, progress, and score history

8. Tools & Technology Stack

This section lists the exact tools and technologies used in the current implementation, alongside their actual status — distinguishing what is actively used in the running system from what was installed but not yet utilized, or is still planned.

8.1 Frontend Technologies

Layer	Technology / Tool	Version / Detail	Status
Framework	React	v18, via Vite build tool	Implemented
Styling	Tailwind CSS	v4 (via @tailwindcss/vite)	Implemented
Routing	React Router DOM	Client-side routing across pages	Implemented
HTTP Client	Axios	With JWT interceptor + 401 auto-logout	Implemented
Animation	Framer Motion	Stat cards, score ring, progress bar animations	Implemented
Icons	lucide-react	UI iconography throughout	Implemented
Charting	Recharts	Installed for future expanded chart views	Installed, not yet used

8.2 Backend Technologies

Layer	Technology / Tool	Version / Detail	Status
Framework	ASP.NET Core Web API	v8 (.NET 8)	Implemented
Language	C#	—	Implemented
ORM	Entity Framework Core	with Npgsql.EntityFrameworkCore.PostgreSQL provider	Implemented
Authentication	JWT Bearer Authentication	Microsoft.AspNetCore.Authentication.JwtBearer	Implemented
Password Hashing	BCrypt.Net-Next	One-way password hashing	Implemented
API Documentation	Swagger / OpenAPI (Swashbuckle)	Auto-generated interactive API docs	Implemented

8.3 Database & Data Layer

Layer	Technology / Tool	Version / Detail	Status
RDBMS	PostgreSQL	v18	Implemented

Layer	Technology / Tool	Version / Detail	Status
Admin Tool	pgAdmin 4	GUI for schema/query management	Implemented
Schema	Normalized relational schema	10 tables (see Section 9)	Implemented

8.4 Machine Learning Service

Layer	Technology / Tool	Version / Detail	Status
Runtime	Python	v3.10+	Implemented
Web Framework	FastAPI	Serves prediction/recommendation endpoints	Implemented
Server	Uvicorn	ASGI server	Implemented
ML Library	scikit-learn	RandomForestRegressor (2 trained models)	Implemented
Data Handling	pandas, numpy	Synthetic dataset generation & feature prep	Implemented
Model Persistence	joblib	Serializes trained models to .pkl files	Implemented

8.5 Development & Testing Tools

Layer	Technology / Tool	Version / Detail	Status
IDE (Backend)	Visual Studio 2022	Primary backend development environment	Implemented
Editor (Frontend/ML)	VS Code	Frontend and Python development	Implemented
API Testing	Swagger UI	Manual endpoint testing during development	Implemented
Version Control	Git	Source control (recommended for submission)	Recommended

8.6 Hardware Requirements (Development Environment)

Component	Minimum Requirement
Processor	Dual-core 2.0 GHz or higher
RAM	8 GB minimum (16 GB recommended, for running DB + 2 backends + frontend concurrently)
Storage	10 GB free space (project, dependencies, Python venv, node_modules)

Component	Minimum Requirement
OS	Windows 10/11, macOS, or Linux
Network	Local network access between the 3 running services (default: all on localhost)

9. Data Requirements (Database Schema Summary)

The system's PostgreSQL database is normalized into 10 tables, summarized below. This schema is already implemented and deployed.

Table	Purpose
users	Stores user accounts: credentials, education background, and experience
skills	Master catalog of all skills, categorized by domain
userskills	Skills a specific user has, with self-rated proficiency level
careerroles	Master catalog of target job roles, with industry, salary, and demand level
roleskills	Maps each role to its required skills, tagged with an importance level
roadmaps	A generated roadmap instance for a specific user and target role
roadmapsteps	Individual ordered steps within a roadmap, one per missing skill
learningresources	Catalog of learning materials (videos, articles, courses) mapped to skills
userprogress	Tracks completion status (Not Started / In Progress / Completed) per roadmap step, per user
employabilityscores	Historical record of every calculated employability score, with timestamp

All foreign-key relationships use cascading deletes where a child record is meaningless without its parent (e.g., deleting a user removes their skills, roadmaps, and scores), preserving referential integrity per NFR-11.

10. Appendix

10.1 Glossary

Term	Definition
Skill Gap	The set of skills required for a target role that a user does not currently possess
Roadmap	An ordered, personalized sequence of skills (and estimated learning time) generated to close a skill gap
Employability Score	A 0–100% indicator of how ready a user is for a target role, computed both by rule-based weighting and by a trained ML model
Readiness Level	A categorical label (Job Ready / Almost Ready / Needs Improvement / Beginner Stage) derived from the employability score
JWT	JSON Web Token — a signed token used to authenticate API requests after login
Synthetic Data	Programmatically generated training data, created from logical rules and randomness, used when real historical data does not exist
Microservice	An independently deployable service (here, the Python ML service) communicating with the main backend over REST

10.2 Full References

Existing systems reviewed for the comparative analysis in Section 3:

LinkedIn Learning — Career Explorer & Skill Assessments <https://www.linkedin.com/learning/>

Coursera Career Academy <https://www.coursera.org/career-academy>

Career.io — Career Pathways <https://career.io/career-pathways>

roadmap.sh — Developer Learning Roadmaps <https://roadmap.sh>

Resumly — Skills Gap Analyzer <https://www.resumly.ai/skills-gap-analyzer>

Prosumely — Career Roadmap Generator & Skill Gap Analyzer <https://www.prosumely.com/career-tools/career-roadmap-generator>

Fuel50 / iMocha — via Quantum Workplace's Best Career Pathing Tools comparison <https://www.quantumworkplace.com/future-of-work/best-career-pathing-tools>